Tài liệu đặc tả yêu cầu phần mềm (SRS) - SQL Hub

Phiên bản 2.2 (dùng boilerplate Brocoders, cập nhật 27/10/2026) | Nền tảng học SQL song ngữ Việt - Anh | Nhóm 5 người (1 Lead + 4 thành viên)

1. Giới thiệu

1.1 Mục đích

Tài liệu này mô tả yêu cầu, kiến trúc, phạm vi và phân công công việc của dự án SQL Hub, một trang web giúp người học làm quen và luyện tập SQL qua bài học, trắc nghiệm và bài thực hành (lab) chạy trực tiếp trên trình duyệt. Tài liệu là cơ sở để cả nhóm thống nhất trước khi code.

1.2 Phạm vi

SQL Hub cho phép người học: đăng nhập nhanh, học theo lộ trình bài học, làm trắc nghiệm, viết và chạy câu SQL trên dữ liệu mẫu, được hệ thống chấm tự động, theo dõi tiến độ, và dùng giao diện tiếng Việt hoặc tiếng Anh. Quản trị viên quản lý nội dung bài học, trắc nghiệm và lab.

1.3 Yêu cầu "AI first"

Dự án phải áp dụng AI first. Nhóm hiểu theo hai mức:

- **Mức bắt buộc:** dùng AI trong quy trình làm việc (viết code, sinh bài tập, vẽ diagram, review PR) và ghi lại bằng chứng sử dụng.
- **Mức điểm cộng:** tích hợp AI vào sản phẩm, ví dụ trợ lý giải thích lỗi SQL và gợi ý hướng làm mà không đưa đáp án.

Câu hỏi cần xác nhận với người hướng dẫn nằm ở mục 14.

1.4 Thuật ngữ

Thuật ngữ	Ý nghĩa
Lab	Bài thực hành: đề bài + dữ liệu mẫu + đáp án để chấm
Sandbox	Môi trường cô lập để chạy câu SQL của người học
Lead	Trưởng nhóm kỹ thuật, chịu trách nhiệm khung dự án và review
P1 / P2 / P3	Mức ưu tiên: bắt buộc / nên có / điểm cộng

2. Mô tả tổng quan

2.1 Đối tượng người dùng

Vai trò	Mô tả	Quyền chính
Khách	Chưa đăng nhập	Xem danh sách bài học, đọc bài mở đầu

Vai trò	Mô tả	Quyền chính
Học viên	Đã đăng nhập	Học, làm trắc nghiệm, làm lab, xem tiến độ
Quản trị viên	Nhóm vận hành	Quản lý nội dung, xem thống kê

2.2 Giả định và ràng buộc

- Công nghệ: Next.js (React), NestJS, PostgreSQL, dựng trên cặp boilerplate của Brocoders.
- Tách hai repo: `sql-hub-frontend` và `sql-hub-backend`.
- Giao diện song ngữ vi/en. Nội dung bài học ưu tiên tiếng Việt, tiếng Anh cho các bài đầu (chờ quyết định ở mục 14).
- Thời gian thực hiện dự kiến 6 tuần.
- Người học chỉ được chạy câu truy vấn đọc (SELECT) trong sandbox ở giai đoạn đầu.

2.3 Ngoài phạm vi

Các việc sau không làm trong 6 tuần, để tránh bị đòi thêm:

- Thanh toán, gói học trả phí.
- Ứng dụng di động riêng (giao diện web dùng được trên điện thoại là đủ).
- Chạy lệnh ghi hoặc thay đổi cấu trúc bảng (INSERT, UPDATE, DELETE, CREATE) trong sandbox.
- Chứng chỉ hoàn thành khóa học, diễn đàn hoặc bình luận giữa học viên.
- Gửi email nhắc học.

3. Kiến trúc và công nghệ

3.1 Tổng quan

- **Frontend (Next.js + TypeScript):** có sẵn shadcn/ui và Tailwind CSS (hỗ trợ dark mode), i18next, React Query, React Hook Form và luồng đăng nhập. Cần thêm trình soạn thảo SQL (CodeMirror hoặc Monaco).
- **Backend (NestJS + TypeScript):** REST API, TypeORM, Swagger, nestjs-i18n, đăng nhập và phân quyền Admin/User có sẵn. Cần thêm đăng nhập GitHub. Token trả trong nội dung phản hồi và gửi qua header Authorization (không dùng cookie), có refresh token.
- **Cơ sở dữ liệu (PostgreSQL):** lưu người dùng, nội dung, tiến độ, kết quả nộp bài. ORM: TypeORM (có sẵn trong boilerplate).
- **Sandbox SQL:** hai lựa chọn, chốt ở Tuần 1:
    - (A) Chạy SQLite/PGlite ngay trên trình duyệt, an toàn cho server, không tốn tài nguyên. Lưu ý: bước Nộp bài vẫn phải được chấm lại trên server, vì đáp án không được để ở phía trình duyệt của người học.
    - (B) Chạy trên một schema PostgreSQL riêng, tài khoản chỉ đọc, giới hạn thời gian và số dòng. Cần làm kỹ phần bảo mật.
- **Hạ tầng:** Docker Compose cho môi trường dev, GitHub Actions cho CI.

3.2 Liên kết hai repo

- Backend xuất tài liệu OpenAPI. Frontend sinh kiểu TypeScript từ file này (`npm run gen:api`), nên khi backend đổi API thì frontend báo lỗi ngay lúc build. Boilerplate chưa có sẵn lệnh này, Lead thêm ở Tuần 1 (hoặc bỏ nếu nhóm không dùng).
- Vì hai repo chạy ở hai địa chỉ khác nhau, cần thêm địa chỉ frontend vào danh sách origin được phép của backend. Vì boilerplate dùng header Authorization thay cho cookie, nên không cần dùng chung tên miền gốc hay proxy `/api` để cookie đăng nhập hoạt động.

3.3 Bộ khung dự án (đã chọn)

Nhóm đã chọn tạo repo từ cặp template `brocoders/nestjs-boilerplate` và `brocoders/extensive-react-boilerplate` để tiết kiệm thời gian dựng khung. Cặp này đã có đăng nhập Google, phân quyền, i18n, Swagger, Docker, CI và test. Chưa có đăng nhập GitHub, tiếng Việt và toàn bộ nghiệp vụ học SQL. Frontend của boilerplate là Next.js, nên dự án dùng Next.js thay cho React + Vite. Backend có lần push gần nhất ngày 2/5/2026. Frontend dùng giấy phép MIT. Trước khi tạo repo, Lead cần kiểm tra ngày commit của repo frontend, phiên bản Node, NestJS, Next.js và giấy phép của backend.

Việc của Lead trên bộ khung này nằm ở mục 9.1.

4. Yêu cầu chức năng

4.1 Xác thực và tài khoản (FR-AUTH)

Mã	Yêu cầu	Ưu tiên
FR-AUTH-01	Đăng nhập bằng Google (OAuth 2.0), dùng module có sẵn của boilerplate	P1
FR-AUTH-02	Đăng nhập bằng GitHub (OAuth 2.0), tự thêm theo cách module Google	P1
FR-AUTH-03	Phiên đăng nhập bằng JWT có làm mới token, dùng cơ chế của boilerplate (token gửi qua header Authorization)	P1
FR-AUTH-04	Đăng xuất	P1
FR-AUTH-05	Trang hồ sơ: tên, ảnh, ngôn ngữ ưa thích	P2
FR-AUTH-06	Đăng nhập email và mật khẩu (boilerplate đã có sẵn, nhóm chọn giữ hay bỏ)	P3

4.2 Đa ngôn ngữ (FR-I18N)

Mã	Yêu cầu	Ưu tiên
FR-I18N-01	Giao diện có hai ngôn ngữ vi và en, đường dẫn dạng `/vi` và `/en`	P1
FR-I18N-02	Nút đổi ngôn ngữ, ghi nhớ lựa chọn	P1

Mã	Yêu cầu	Ưu tiên
FR-I18N-03	Nội dung bài học, trắc nghiệm, lab lưu theo từng ngôn ngữ trong cơ sở dữ liệu	P1
FR-I18N-04	Công cụ kiểm tra thiếu key vi/en chạy trong CI	P2

4.3 Bài học (FR-LESSON)

Mã	Yêu cầu	Ưu tiên
FR-LESSON-01	Danh sách khóa học và chương theo lộ trình (3 khóa: SQL cơ bản, JOIN và nhóm dữ liệu, truy vấn con và thiết kế bảng)	P1
FR-LESSON-02	Trang bài học hiển thị nội dung Markdown, có khối code SQL	P1
FR-LESSON-03	Đánh dấu hoàn thành bài, hiển thị thanh tiến độ	P1
FR-LESSON-04	Tìm kiếm bài học theo từ khóa	P2
FR-LESSON-05	Sơ đồ quan hệ bảng (ERD) minh họa cho dữ liệu mẫu	P3

4.4 Trắc nghiệm (FR-QUIZ)

Mã	Yêu cầu	Ưu tiên
FR-QUIZ-01	Mỗi bài hoặc chương có bộ câu hỏi một đáp án đúng	P1
FR-QUIZ-02	Chấm tự động, hiển thị điểm và giải thích từng câu	P1
FR-QUIZ-03	Lưu lịch sử làm bài và điểm cao nhất	P2
FR-QUIZ-04	Câu hỏi nhiều đáp án, xáo trộn thứ tự câu	P3

4.5 Lab thực hành SQL (FR-LAB)

Mã	Yêu cầu	Ưu tiên
FR-LAB-01	Trang lab có đề bài, mô tả bảng dữ liệu mẫu, trình soạn thảo SQL	P1
FR-LAB-02	Nút Chạy: thực thi câu SQL trong sandbox và hiển thị kết quả dạng bảng hoặc thông báo lỗi	P1
FR-LAB-03	Nút Nộp bài: so sánh kết quả với đáp án (so sánh tập kết quả, không so sánh chuỗi), báo đạt hoặc chưa đạt	P1
FR-LAB-04	Gợi ý theo cấp độ (xem gợi ý không lộ đáp án)	P2
FR-LAB-05	Lưu bản nháp câu truy vấn và lịch sử nộp	P2
FR-LAB-06	Giới hạn: chỉ SELECT, tối đa 5 giây, tối đa 1.000 dòng trả về, giới hạn số lần chạy mỗi phút (boilerplate không bundle sẵn rate	P1

Mã	Yêu cầu	Ưu tiên
	limiting)	

4.6 Tiến độ và thống kê (FR-PROG)

Mã	Yêu cầu	Ưu tiên
FR-PROG-01	Trang tổng quan của học viên: bài đã học, điểm trắc nghiệm, lab đã đạt	P1
FR-PROG-02	Chuỗi ngày học liên tiếp, huy hiệu	P3

4.7 Quản trị nội dung (FR-ADMIN)

Mã	Yêu cầu	Ưu tiên
FR-ADMIN-01	Phân quyền admin, chỉ admin vào được trang quản trị	P1
FR-ADMIN-02	Thêm, sửa, xóa bài học, câu hỏi, lab (hai ngôn ngữ)	P2
FR-ADMIN-03	Nhập nội dung hàng loạt từ file Markdown hoặc JSON (dữ liệu seed)	P2
FR-ADMIN-04	Thống kê số người học, tỷ lệ đạt từng lab	P3

4.8 Trợ lý AI trong sản phẩm (FR-AI, điểm cộng)

Mã	Yêu cầu	Ưu tiên
FR-AI-01	Ô chat cạnh lab: học viên gửi câu truy vấn và lỗi, AI giải thích và gợi ý hướng sửa	P3
FR-AI-02	AI không đưa đáp án hoàn chỉnh của lab (ràng buộc trong prompt hệ thống)	P3
FR-AI-03	Giới hạn số lượt hỏi mỗi người mỗi ngày, ẩn khóa API ở backend	P3

5. Yêu cầu phi chức năng

Mã	Nhóm	Yêu cầu
NFR-01	Hiệu năng	Trang tải lần đầu dưới 3 giây trên mạng thông thường. API thường dưới 500 ms
NFR-02	Bảo mật	Không lưu mật khẩu nếu chỉ dùng OAuth. Token gửi qua header Authorization, access token hết hạn sau 15 phút theo mặc định, có refresh token. Kiểm tra dữ liệu đầu vào ở backend

Mã	Nhóm	Yêu cầu
NFR-03	Bảo mật sandbox	Tài khoản chỉ đọc, chặn lệnh ghi và lệnh hệ thống, giới hạn thời gian và dòng, không truy cập bảng của hệ thống
NFR-04	Bí mật	Khóa API, secret OAuth nằm trong biến môi trường, không commit. Có hook hoặc bước CI chặn secret
NFR-05	Khả dụng	Giao diện dùng được trên điện thoại và máy tính
NFR-06	Chất lượng mã	ESLint, Prettier, TypeScript strict, mỗi PR có ít nhất 1 người review
NFR-07	Kiểm thử	Backend có test cho chấm bài và xác thực. Frontend có test cho luồng đăng nhập và làm lab (nếu kịp)
NFR-08	Triển khai	Chạy được toàn bộ hệ thống bằng `docker compose up`. Có hướng dẫn trong README
NFR-09	Tài liệu	API có Swagger. Mỗi repo có README, CLAUDE.md và hướng dẫn đóng góp

6. Mô hình dữ liệu (mức khái quát)

Bảng	Trường chính	Ghi chú
users	id, email, name, avatar, provider, provider_id, role, locale	provider: google hoặc github
courses	id, slug, order, is_published	
course_translations	course_id, locale, title, description	Một dòng cho mỗi ngôn ngữ
lessons	id, course_id, slug, order	
lesson_translations	lesson_id, locale, title, content_md	
quizzes, quiz_questions, question_options	gắn với bài hoặc chương, có đáp án đúng	Bản dịch lưu bảng riêng
quiz_attempts	id, user_id, quiz_id, score, answers, created_at	
labs	id, lesson_id, dataset_id, expected_query, order_sensitive	Bản dịch: đề bài, gợi ý
datasets	id, name, setup_sql	Bảng và dữ liệu mẫu cho sandbox
lab_submissions	id, user_id, lab_id, query, passed, runtime_ms,	

Bảng	Trường chính	Ghi chú
	created_at	
lesson_progress	user_id, lesson_id, completed_at	

7. API chính (dự kiến)

Mọi endpoint dưới đây nằm sau tiền tố `/api/v1` của boilerplate. Endpoint GitHub và các module nghiệp vụ do nhóm tự thêm.

Nhóm	Endpoint	Mô tả
Auth	POST /auth/google/login, POST /auth/github/login, GET /auth/me, POST /auth/refresh, POST /auth/logout	Đăng nhập OAuth (frontend gửi token của nhà cung cấp lên backend), làm mới token và phiên
Nội dung	GET /courses, GET /lessons/:slug	Có tham số `locale`
Tiến độ	POST /lessons/:id/complete, GET /progress	
Trắc nghiệm	GET /quizzes/:id, POST /quizzes/:id/submit	
Lab	GET /labs/:id, POST /labs/:id/run, POST /labs/:id/submit	Run chỉ trả kết quả, submit chấm điểm
Admin	CRUD /admin/lessons, /admin/quizzes, /admin/labs	Chỉ role admin
AI	POST /ai/explain	Điểm cộng

8. Quy trình AI first của nhóm

1. Dùng Claude Code (hoặc công cụ được chỉ định) làm công cụ chính. Nếu người hướng dẫn chỉ định Copilot hay Cursor, giữ nguyên các ý tưởng bên dưới và đặt theo cách công cụ đó hỗ trợ.
2. Mỗi repo có tệp `CLAUDE.md` mô tả cấu trúc, lệnh chạy, quy ước code và những điều không được làm.
3. Dùng `claude-howto` làm tài liệu tham khảo. Chọn mẫu cần thiết, sửa cho đúng dự án, rồi commit thư mục `.claude/` để cả nhóm dùng chung. Không sao chép nguyên si.
4. Bộ công cụ dùng chung:
    - Skill viết bài học SQL song ngữ theo mẫu.
    - Skill review PR theo checklist của nhóm.
    - Lệnh `/new-feature` để dựng nhanh khung một tính năng.
    - Lệnh `/ai-log` để ghi bằng chứng dùng AI.
    - Hook chặn commit chứa secret.
5. **Bằng chứng AI:** mỗi PR có mục "AI đã giúp gì" (prompt chính, phần AI sinh ra, phần người sửa). Tổng hợp vào `docs/ai-log.md` mỗi tuần.

6. AI hỗ trợ nhưng người chịu trách nhiệm: mọi mã do AI sinh ra phải được người trong nhóm đọc hiểu và review trước khi merge.
7. Dùng AI để sinh nội dung: đề lab, câu trắc nghiệm, bản dịch vi/en. Người phụ trách nội dung kiểm tra lại độ đúng của SQL và đáp án bằng cách chạy thật.

9. Phân công công việc

Nhóm 5 người: 1 Lead và 4 thành viên (M1 đến M4). Mỗi người có thư mục riêng để hạn chế xung đột khi merge. Tên người thật điền vào cột "Người thực hiện".

Vai trò	Người thực hiện	Repo	Phạm vi chính
Lead	(điền tên)	Cả hai	Khung dự án, kiến trúc, xác thực, review PR, quy trình AI, tích hợp
Đặng Nguyễn Phúc Khang	(điền tên)	Frontend + Backend	Module bài học, khóa học, tiến độ, trang bài học
Đỗ Trung Tú	(điền tên)	Frontend + Backend	Module trắc nghiệm, chấm điểm, lịch sử làm bài
Lê Văn Minh	(điền tên)	Frontend + Backend	Module lab, trình soạn thảo SQL, sandbox, chấm bài
Nguyễn Hoàng Quốc Thắng	(điền tên)	Frontend + Backend	Trang quản trị, dữ liệu seed hai ngôn ngữ, kiểm tra key vi/en, trang tổng quan, hồ sơ

9.1 Chi tiết từng vai trò

Lead

- Chốt kiến trúc trong `docs/architecture.md`: cách chạy và chấm bài trên sandbox, cách frontend cất token đăng nhập (ORM đã chốt là TypeORM).
- Tạo hai repo từ hai template của Brocoders (dùng Use this template, không dùng Fork), đặt tên đúng, chạy thử cả hai bằng Docker Compose và npm run dev, dọn bớt phần không dùng (gửi mail, upload file, quản lý user).
- Làm xác thực: cấu hình Google có sẵn, thêm GitHub ở backend và nút đăng nhập ở frontend, kiểm tra phân quyền.
- Thêm bộ chuỗi tiếng Việt (vi) vào i18n của cả hai repo, kiểm tra nút đổi ngôn ngữ.
- Viết entity TypeORM, migration ban đầu và dữ liệu seed mẫu. Thêm lệnh sinh kiểu TypeScript từ OpenAPI (gen:api) cho frontend.
- Dựng bộ khung AI first: `CLAUDE.md`, skill, lệnh, hook (mục 8).
- Làm một tính năng mẫu hoàn chỉnh từ database đến giao diện (đề xuất: xem danh sách bài học) để cả nhóm sao chép.
- Viết checklist "thêm một tính năng trong 7 bước" và quy trình nhánh, PR.

- Họp kickoff 60 phút rồi giao task. Từ Tuần 2 chủ yếu review PR, gỡ vướng, tích hợp.
- Triển khai bản demo và chuẩn bị báo cáo, thuyết trình.

Đặng Nguyễn Phúc Khang - Nội dung và bài học (FR-LESSON)

- Backend: module courses, lessons, lesson_progress.
- Frontend: trang danh sách khóa học, trang bài học hiển thị Markdown, thanh tiến độ, tìm kiếm.
- Soạn nội dung các bài đầu (dùng AI sinh bản nháp, kiểm tra lại).

Đỗ Trung Tú - Trắc nghiệm (FR-QUIZ)

- Backend: module quizzes, chấm điểm, lưu quiz_attempts.
- Frontend: giao diện làm bài, kết quả, giải thích, lịch sử.
- Soạn bộ câu hỏi cho các chương (vi/en).

Lê Văn Minh - Lab và sandbox (FR-LAB, NFR-03) - phần khó nhất, nên bắt đầu sớm

- Backend: module labs, chạy truy vấn trong sandbox, chấm bài bằng so sánh tập kết quả, giới hạn thời gian và dòng.
- Frontend: trang lab, trình soạn thảo SQL, bảng kết quả, thông báo lỗi, gợi ý.
- Chuẩn bị các bộ dữ liệu mẫu (`datasets`) và đáp án.
- Viết test cho chấm bài và test bảo mật sandbox.

Nguyễn Hoàng Quốc Thắng - Admin, dữ liệu và i18n (FR-ADMIN, FR-I18N, FR-PROG)

- Backend: module admin (CRUD nội dung hai ngôn ngữ), nhập hàng loạt.
- Frontend: trang quản trị, trang tổng quan học viên (FR-PROG-01, lấy dữ liệu từ module tiến độ của M1, trắc nghiệm của M2 và lab của M3), trang hồ sơ.
- Công cụ kiểm tra thiếu key vi/en trong CI.
- Gom và kiểm tra dữ liệu seed của M1, M2, M3.
- Nếu còn thời gian: trợ lý AI trong sản phẩm (FR-AI) phối hợp với M3.

9.2 Quy tắc chia việc

- Mỗi module có thư mục riêng ở cả hai repo (ví dụ `src/modules/quizzes` ở backend và `src/features/quizzes` ở frontend). Ai làm module nào chỉ sửa thư mục đó. Tên thư mục ở frontend là ví dụ, Lead chốt theo cấu trúc thật của repo sau khi tạo. Sửa phần chung phải báo Lead.
- Backend của mỗi module phải có Swagger trước, để frontend sinh kiểu và làm song song.
- Mỗi người tự làm cả backend lẫn frontend cho module của mình (làm theo chiều dọc), tránh phải chờ nhau.

10. Kế hoạch theo tuần

Tuần	Mục tiêu	Lead	Đặng Nguyễn Phúc Khang	Đỗ Trung Tú	Lê Văn Minh	Nguyễn Hoàng Quốc Thắng
1	Dựng	Fork và chạy thử	Đọc khung,	Đọc khung,	Nghiên cứu	Đọc khung,

Tuần	Mục tiêu	Lead	Đặng Nguyễn Phúc Khang	Đỗ Trung Tú	Lê Văn Minh	Nguyễn Hoàng Quốc Thắng
	khung	boilerplate, dọn bớt, thêm GitHub và tiếng Việt, tính năng mẫu, bộ AI	soạn đề cương nội dung	soạn đề cương câu hỏi	sandbox, thử nghiệm A/B	thiết kế dữ liệu seed
2	Làm tính năng nền	Review, gỡ vướng	Bài học (API + giao diện)	Trắc nghiệm (API)	Sandbox chạy được SELECT	Dữ liệu seed, i18n CI
3	Hoàn thiện tính năng P1	Review, tích hợp	Tiến độ, trang bài học	Trắc nghiệm (giao diện, chấm điểm)	Lab: chạy và nộp bài	Trang tổng quan học viên
4	Tính năng P2	Review, bảo mật, hiệu năng	Tìm kiếm, nội dung các chương	Lịch sử, giải thích	Gợi ý, nháp, test sandbox	Trang quản trị, nhập hàng loạt
5	Nội dung và điểm cộng	Triển khai bản demo	Hoàn thiện nội dung	Hoàn thiện câu hỏi	Thêm lab, trợ lý AI (nếu kịp)	Kiểm tra song ngữ, thống kê
6	Kiểm thử và bàn giao	Báo cáo, thuyết trình, tổng hợp AI log	Sửa lỗi	Sửa lỗi	Sửa lỗi, test	Sửa lỗi, tài liệu

Cột mốc: cuối Tuần 1 chạy được khung và đăng nhập. Cuối Tuần 3 hoàn thành các tính năng P1. Cuối Tuần 5 có bản demo trên mạng. Tuần 6 chỉ sửa lỗi và hoàn thiện.

11. Quy trình làm việc chung

1. Nhánh: `main` (ổn định), `develop` (tích hợp), nhánh tính năng đặt tên `feat/<module>-<mô tả>`.
2. Mỗi tính năng làm theo checklist 7 bước của Lead: (1) tạo nhánh, (2) viết schema và migration, (3) viết API và Swagger, (4) chạy `gen:api` ở frontend, (5) làm giao diện có đủ chuỗi vi/en, (6) viết test, (7) mở PR kèm mục "AI đã giúp gì".
3. PR cần ít nhất 1 người duyệt (Lead hoặc thành viên được chỉ định). CI phải xanh trước khi merge.
4. Họp đứng 15 phút mỗi hai ngày. Họp tổng kết và demo vào cuối mỗi tuần.
5. Công việc quản lý bằng bảng GitHub Projects, mỗi task gắn mã yêu cầu (ví dụ FR-LAB-03).
6. Commit theo quy ước Conventional Commits.

12. Tiêu chí nghiệm thu

- Học viên đăng nhập được bằng Google và GitHub.

- Có ít nhất 3 khóa, mỗi khóa tối thiểu 5 bài, mỗi bài 3 câu trắc nghiệm và mỗi khóa 2 lab (con số tạm, nhóm chốt ở Tuần 1); các bài đầu có bản tiếng Anh.
- Lab chạy và chấm đúng, câu lệnh ghi hoặc quá thời gian bị chặn.
- Đổi ngôn ngữ vi/en không mất ngữ cảnh trang.
- Tiến độ của học viên được lưu và hiển thị đúng.
- Hệ thống chạy bằng `docker compose up` và có bản demo trực tuyến.
- Có `docs/ai-log.md` và các PR ghi rõ phần AI đã hỗ trợ.

13. Rủi ro và cách giảm thiểu

Rủi ro	Mức	Cách giảm thiểu
Sandbox bị lợi dụng (chạy lệnh nguy hiểm, truy vấn nặng)	Cao	Chọn phương án A (chạy trên trình duyệt) hoặc tài khoản chỉ đọc, giới hạn thời gian và dòng, có test bảo mật
Lỗi CORS hoặc cất token sai cách giữa hai repo	Trung bình	Thêm địa chỉ frontend vào origin được phép của backend (hoặc dùng proxy `/api`), kiểm tra sớm ở Tuần 1
Làm chậm vì chờ nhau	Trung bình	Làm theo chiều dọc, Swagger trước, thư mục riêng
Nội dung do AI sinh có SQL sai	Trung bình	Chạy thật mọi đáp án, người phụ trách duyệt
Quá tải việc ở Lead	Trung bình	Đã dùng boilerplate, giao review cho một thành viên khi cần
Dịch toàn bộ nội dung tốn thời gian	Thấp	Chỉ dịch giao diện và các bài đầu, dùng AI dịch rồi người duyệt
Phụ thuộc vào một công cụ AI	Thấp	Ghi quy trình theo ý tưởng chung, dễ chuyển sang công cụ khác

14. Các điểm cần chốt

1. **AI first:** chỉ cần dùng AI trong quá trình làm (mức bắt buộc), hay cần tích hợp AI vào sản phẩm? Nên hỏi lại người hướng dẫn.
2. **Công cụ AI được chỉ định:** Claude Code, Copilot hay Cursor?
3. **Frontend:** đã chốt dùng Next.js theo boilerplate Brocoders.
4. **Boilerplate:** đã chốt dùng cặp template Brocoders. Lead thêm đăng nhập GitHub, bộ chuỗi tiếng Việt và dọn bớt phần không dùng; cần kiểm tra giấy phép và phiên bản trước khi tạo repo.
5. **Đăng nhập email và mật khẩu:** có giữ không, hay chỉ dùng Google/GitHub? Đề xuất: giữ, vì boilerplate đã có sẵn.
6. **Phạm vi dịch:** toàn bộ nội dung sang tiếng Anh, hay chỉ giao diện và các bài đầu?
7. **Sandbox:** phương án A (trình duyệt) hay B (schema PostgreSQL riêng)?
8. **ORM:** đã chốt TypeORM (có sẵn trong boilerplate).
9. **Thành viên:** điền tên vào mục 9 và xác nhận số tuần thực hiện.

15. Lịch sử phiên bản

Phiên bản	Ngày	Thay đổi
2.2	27/10/2026	Sửa theo tài liệu thực tế của boilerplate: shadcn/ui, token qua header Authorization, API có tiền tố /api/v1. Bỏ nội dung trùng ở mục 3.3, chốt người làm trang tổng quan học viên (M4), thêm mục ngoài phạm vi
2.1	(chưa ghi)	Chuyển sang dùng cặp boilerplate Brocoders, chốt TypeORM và Next.js
2.0	(chưa ghi)	Bản viết lại đầy đủ SRS và phân công cho 5 người